FORMADORES { IT }

Kubernetes

5. Kubectl

Índice

Introducción a kubectl. Aspectos generales

Estructura de un comando

Comandos para listar recursos

Filtros y personalización de la respuesta

Comandos para crear recursos

Comandos para eliminar recursos

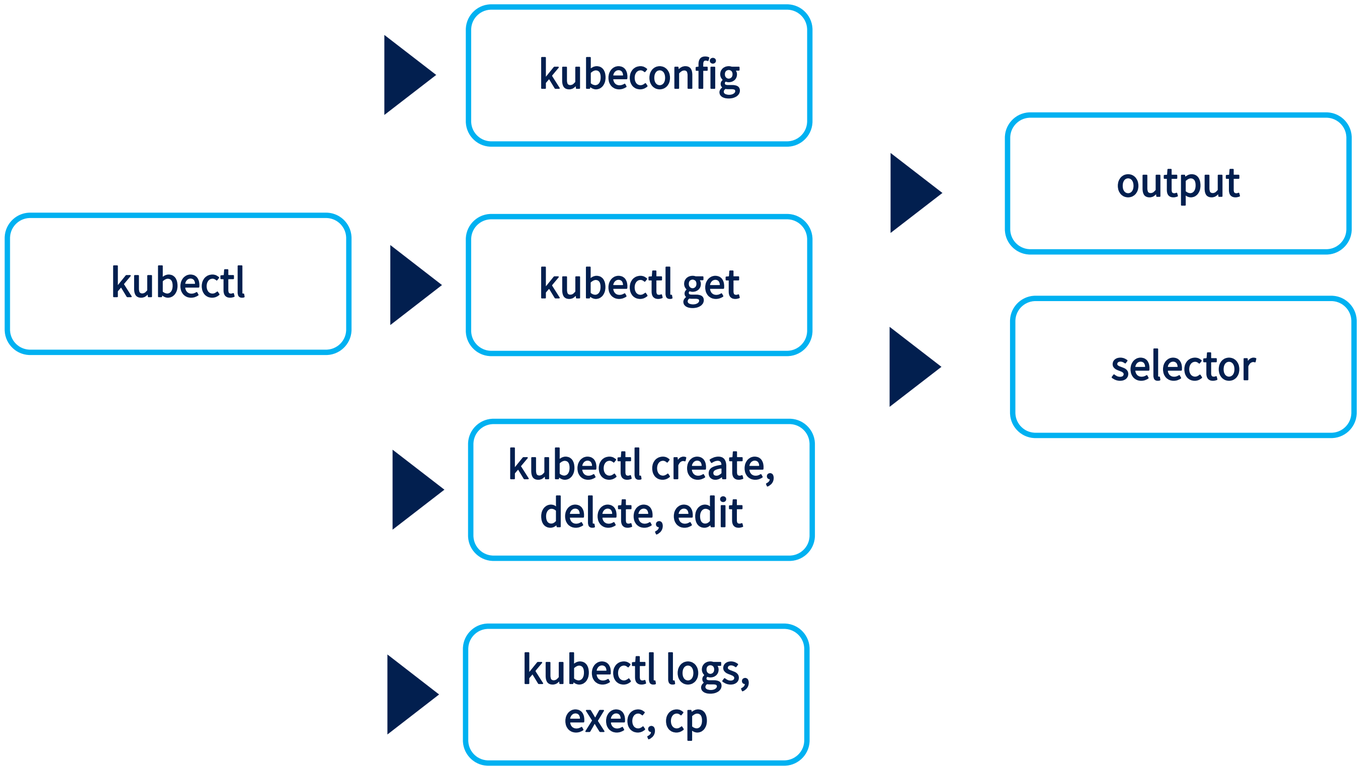
Comandos para interactuar con pods

Plugins para kubectl

Objetivos

- Saber instalar y configurar kubectl para cualquier clúster
- Obtener los datos exactos que buscamos de los recursos
- Crear recursos más eficientemente
- Ver cómo podemos interactuar con los contenedores

Mapa Conceptual



kubectl

kubectl es la herramienta fundamental para interactuar con Kubernetes.

Es una herramienta de línea de comandos que nos permite enviar y recibir información de nuestro clúster.

Hay otras formas de hacerlo, como interfaces web y de escritorio por ejemplo, pero kubectl es la única que ofrece todas las posibilidades. No es sustituible.

Toda la documentación y recursos que encontrarás se basan en esta herramienta.

Instalación

Hay multitud de métodos disponibles para instalar kubectl en cualquier sistema:

- Descargar el ejecutable para Linux, Mac o Windows
- apt, yum o snap en Linux
- Homebrew en Mac
- Chocolatey en Windows

Para comprobar si lo tienes instalado y en qué versión:

```
kubectl versión --client
```

kubeconfig

Instalar kubectl no es suficiente para poder conectarnos a nuestro clúster. Tenemos que configurarlo con la URL del clúster y si es necesario, con el certificados TLS y los datos para autenticarse (lo será en cualquier entorno que no sea local).

Todos estos datos se almacenan en el archivo llamado kubeconfig. Es un archivo de configuración básico que vas a utilizar continuamente.

Es una archivo YAML con 3 apartados:

1. Clusters
2. Contexts
3. Users

kubeconfig

Clusters es una lista con las URL y los certificados TLS de todos los clúster a los que tienes acceso.

Users es una lista de los usuarios con los que puedes conectarte a esos clústeres

Y context relaciona un clúster con un usuario.

Efectivamente, puedes tener varios usuarios para un mismo clúster, y también el mismo usuario para distintos clústeres.

kubeconfig

```
apiVersion: v1
kind: Config
clusters:
- cluster:
    certificate-authority-data: ...
    server: https://172.17.0.30:6443
  name: my-cluster
contexts:
- context:
    cluster: my-cluster
    user: admin-user
  name: cluster-admin
current-context: cluster-admin
users:
- name: admin-user
  user:
    client-certificate-data: ...
    client-key-data: ...
```

kubeconfig

Clusters:

- Server: URL al clúster de Kubernetes
- certificate-authority-data: certificado TLS para las comunicaciones con el clúster (en base64)

Users:

- client-certificate-data: Certificado de cliente para el clúster (en base64)
- client-key-data: Clave del clientes (en base64)

Estos parámetros dependen del método de autenticación del clúster. Cada proveedor dará la información a incluir en el kubeconfig para su clúster

kubeconfig

Context:

- cluster: nombre del cluster al que referencia el contexto
- user: nombre del usuario a utilizar en el contexto

Los contextos nos permiten conectarnos a distintos cluster y con distintos user. En todos los comandos de kubectl, puedes añadir el parámetro --context para especificar qué contexto del kubeconfig quieres utilizar.

El campo current-context en el fichero marca el contexto por defecto para kubectl.

kubeconfig

Los archivos kubeconfig no se suelen escribir a mano. Normalmente te lo proporciona el proveedor del clúster.

Además existen comandos en kubectl para configurar el fichero kubecontext sin necesidad de escribir todo a mano, codificar los certificados en base64, etc.

kubectl config set-cluster -> Crear un nuevo elemento en clusters

kubectl config set-user -> nuevo elemento en users

kubectl config set-context -> nuevo contexto

kubectl config use-context -> especificar el contexto por defecto

kubeconfig

El archivo kubeconfig por defecto se encuentra en la ruta ~/.kube/config

También es posible tener varios kubeconfigs, en lugar de varios clústeres y usuarios en el mismo kubeconfig.

En todos los comandos de kubectl puedes añadir el parámetro --kube-config con la ruta del archivo que quieres utilizar.

También es útil definir la variable de entorno KUBECONFIG=<ruta al archivo>. Todos los comandos kubectl en esa terminal utilizarán el archivo deseado en lugar de ~/.kube/config

Parámetros globales

Hay una serie de parámetros de kubectl que podemos utilizar en todos los comandos:

- --context: el context de kubeconfig. Por defecto es el current-context en kubeconfig
- --kube-config: el archivo kubeconfig. Por defecto, el valor de la variable de entorno KUBECONFIG o en su defecto ~/.kube/config
- --namespace o -n: el namespace. Por defecto es default
- --dry-run: No modifica ningún recurso en la ejecución. Simplemente ejecuta las validaciones y nos devuelve la respuesta. útil para hacer pruebas.
- --v=[0-9]: controla la *verbosidad* de la respuesta del comando

Estructura general de los comandos

```
kubectl [command] [TYPE] [NAME] [flags]
```

Por ejemplo:

```
kubectl get deployment nginx -o wide
```

- command: get
- type: deployment. El tipo de objeto que quieres para el commando
- nombre nginx: nombre del objeto en concreto
- flags -o wide: parámetros para el comando

Abreviaturas

- k -> kubectl
- -n -> --namespace
- po -> pod
- deploy -> deployment
- sts -> statefulset
- ds -> daemonset
- svc -> service
- cm -> configmap

Todos los recursos se pueden poner el singular o plural

Comandos para listar recursos

Se puede listar cualquier recurso en un namespace con

```
kubectl get <TYPE>
```

```
kubectl get pods
```

También se pueden listar los recursos de todos los Namespaces con el parámetro --all-namespaces

```
kubectl get pods --all-namespaces
```

Comandos para listar recursos

Añadiendo el nombre de un recurso, nos muestra sólo ese objeto:

```
kubectl get <TYPE> <NAME>
```

Con el parámetro -o (--output) controlamos el tipo de respuesta que recibiremos del clúster.

Por defecto nos devuelve sólo unos pocos datos, que dependen del tipo de objeto que estemos listando.

Hemos visto también -o wide que nos devuelve más información útil sobre los recursos (en los pod, no da por ejemplo la dirección IP y el nodo)

Comandos para listar recursos

Otro output útil es -o yaml. Con él conseguimos todo el archivo YAML del objeto. Si no hemos especificado ningún nombre, nos devolverá un YAML con todos los objetos.

Verás que hay muchos más campos de los que en realidad le hemos pasado a Kubernetes desde nuestros archivos. Kubernetes introduce muchos valores por defecto que no necesitamos especificar pero que son parte del objeto que ha creado. Además también está el campo status que Kubernetes va actualizando para tener constancia del estado del objeto.

También existe -o json para obtener exactamente la misma información que con yaml pero en formato json.

Comandos para listar recursos

Con -o name nos devuelve sólo una lista con los nombres de objetos. Útil si queremos crear scripts ya que la salida no tiene ningún dato extra.

Es posible personalizar el output hasta el infinito, eligiendo qué campos específicos queremos y el formato.

Para elegir los campos se pueden utilizar las opciones -o go-template o -o jsonpath.

Jsonpath es más intuitivo para empezar si no conoces el lenguaje Go.

Comandos para listar recursos

Con este comando podemos listar todas las imágenes de nuestros pods:

```
kubectl get pods --all-namespaces -o
jsonpath="{.items[*].spec.containers[*].image}"
```

Para conocer la ruta hasta el campo que queremos, temenos que mirar la estructura del YAML del objeto.

Tanto jsonpath como go-template permiten crear outputs con prácticamente cualquier formato que busquemos.

Comandos para listar recursos

Por último, otro parámetro que usaremos a menudo es -l o --selector.

Con él podemos filtrar la lista de objetos devuelta gracias a las etiquetas o labels:

```
k -n kube-system get po -l k8s-app=kube-proxy
```

Nos devolverá sólo los pods etiquetados con app=kube-proxy

Personalizando filtros y outputs, podemos crear todo tipo de scripts con kubectl.

Comandos para crear recursos

La mejor estrategia para crear recursos es la que hemos visto hasta ahora. Crear archivos YAML y enviarlos al clúster con kubectl apply. Es la mejor porque de esta forma conservas tus YAML en código y es sencillo replicarlos y editarlos. Por supuesto es algo imprescindible en un sistema de producción.

Además kubectl apply es un comando que sirve tanto para crear como para modificar recursos.

Aparte de este comando, vamos a ver otros que podemos utilizar para crear recursos de forma más rápida que escribiendo el archivo YAML.

Comandos para crear recursos

Con kubectl create podemos crear cualquier tipo de recurso en la línea de comando. No ofrece todas las posibilidades pero es una forma rápida y directa:

```
kubect create deploy test-nginx --image nginx
```

Podemos crear services, configMaps, secrets, etc.

Utiliza kubectl create <TYPE> -h para obtener todas las posibilidades de para cada tipo de recurso.

Comandos para crear recursos

En el caso concreto de los deployments y pods, podemos utilizar también kubectl run para crearlos. Es mejor utilizar kubectl create deployment pero con run obtenemos algunas opciones extra.

Por ejemplo es muy útil para crear pods efímeros para realizar pruebas. Con este comando puedes crear un pod con la imagen de Ubuntu, que abra una terminal en él para poder hacer pruebas, y que una vez salgamos de esa terminal, el pod se elimine automáticamente:

```
kubectl run ubuntu -ti --rm --image ubuntu --restart=Never
```

Comandos para crear recursos

Como en Docker:

- - it crea un terminal en el contenedor y lo hace interactivo
- --rm elimina el pod una vez el proceso principal (el terminal) haya terminado
- --restart=Never hace que no se vuelva a recrear el pod. Es decir, no crea un deployment que lo vuelva a levantar.

Con este tipo de pods efímeros podemos tener una terminal en un pod rápidamente para probar la conectividad por ejemplo.

Comandos para eliminar recursos

Eliminar recursos es muy sencillo:

```
kubectl delete <TYPE> <nombre del recurso>
```

```
kubectl delete service nginx
```

Comandos para editar recursos

Otro comando que es muy útil para modificar rápidamente recursos es edit.

kubectl edit se aplica sobre un recurso que ya existe. Abre un editor de texto en el que podemos editar el YAML del recurso como queramos. Una vez guardado, salimos del editor, y automáticamente Kubernetes modificará el recurso. Además, comprobará que no hay ningún error en el YAML. En caso de errores, reabrirá el editor con un mensaje sobre el error.

```
kubectl edit <TYPE> <nombre del recurso>
```

El editor por defecto en Linux y Mac se controla con la variable de entorno EDITOR

Comandos para crear YAML

kubectl explain <TYPE>

Con este comando conseguimos una referencia completa de las opciones para cualquier objeto, con las explicaciones de cada campo que podemos añadir en el YAML

Además podemos ir bajando en el árbol del YAML con

kuebctl explain <TYPE>.<key>.<key>…

Comandos para crear YAML

Por ejemplo:

kubectl explain deploy.spec.template.spec

Nos devolverá todas las opciones que podemos añadir al spec de los pods de un deployment (containers, volumes, etc.)

Comandos para crear YAML

Con kubectl podemos generar el YAML para cualquier recurso automáticamente, sin necesidad de escribirlo siempre desde 0. Este truco es válido tanto para kubectl create como para kubectl run.

Si añadimos las opciones --dry-run (para que no cree el recurso de verdad) y -o yaml para que nos devuelva la salida en formato YAML, obtendremos el fichero que crearía kubectl con ese comando, pero sin crearlo en realidad.

A continuación, podemos coger ese YAML sencillo y completarlo y guardarlo.

Comandos para crear YAML

```
kubectl create deploy test --image=nginx --dry-run -o yaml
```

Nos devolverá el YAML de un deployment con la imagen de Nginx y nada más.

Podemos hacer esto con cualquier tipo de recurso.

Es una forma rápida de crear YAML con lo más básico paracualquier objeto de Kubernetes (muy útil para configMaps y secrets)

Comandos para cargas de trabajo

Vamos a ver varios comandos para interactuar con Deployments, StatefulSets y DaemonSets.

El primero es un atajo para actualizar el número de réplicas automáticamente (no tiene sentido en DaemonSets):

```
kubectl scale <TYPE>/<nombre del recurso> --replicas 5
```

Por ejemplo:

```
kubectl scale deploy/nginx-deployment--replicas 5
```

Comandos para cargas de trabajo

kubectl expose creará un servicio automáticamente para nuestras cargas de trabajo:

kubectl expose <TYPE>/<nombre del recurso> --port <puerto> --target-port <puerto en los pod>

Por ejemplo:

kubectl expose deploy/nginx-deployment --port 80 --target-port 80

Comandos para cargas de trabajo

kubectl expose creará un servicio automáticamente para nuestras cargas de trabajo:

```
kubectl expose <TYPE>/<nombre del recurso> --port <puerto> --target-port <puerto en los pod>
```

Por ejemplo:

```
kubectl expose deploy/nginx-deployment --port 80 --target-port 80
```

Comandos para cargas de trabajo

kubectl rollout contiene varios comandos para controlar todo respecto a la actualización de Deployments, StatefulSets y DaemonSets.

kubectl rollout status nos mostrará el estado de actualización (cuántas réplicas están actualizadas, si ha terminado o no, etc.)

kubectl rollout status <TYPE>/<nombre del recurso>

kubectl rollout status deploy/nginx-deployment

Comandos para cargas de trabajo

kubectl rollout history lista todas las versiones que se han desplegado de la aplicación. Con el parámetro --revision puedes ver el YAML exacto de cada una de ellas.

```
kubectl rollout history <TYPE>/<nombre del recurso>

kubectl rollout history deploy/nginx-deployment

kubectl rollout history deploy/nginx-deployment --revision 1
```

Comandos para cargas de trabajo

kubectl rollout undo podemos hacer rollback, es decir, volver a una versión anterior.

kubectl rollout undo <TYPE>/<nombre del recurso> --to-revision <número de revision>

kubectl rollout undo deploy/nginx-deployment --to-revision 1

Comandos para pods

Hay varios comandos que se utilizan continuamente y que sólo son válidos para pods.

Por ejemplo uno que hemos utilizado continuamente, kubectl logs, nos mostrará los logs que ha producido el pod.

kubectl logs <nombre del pod>

Con el parámetro -c (--container) podemos controlar de qué contenedor queremos mostrar los logs. Es imprescindible si nuestro pod tiene más de un contenedor.

Con el parámetro -f (--follow) irá actualizando los logs según se generan.

Y finalmente con --since podemos pasarle un intervalo de tiempo para mostrar sólo los últimos logs.

Comandos para pods

Con kubectl exec ejecutaremos un comando dentro de un contenedor:

kubectl exec <nombre del pod> <contenedor> -- <comando>

Si tenemos un solo contenedor podemos ahorrarnos ese parámetro.

Comandos para pods

Con kubectl exec ejecutaremos un comando dentro de un contenedor:

kubectl exec <nombre del pod> <contenedor> -- <comando>

Si tenemos un solo contenedor podemos ahorrarnos ese parámetro.

Una opción muy común es abrir un terminal en el contenedor:

kubectl exec -it <nombre del pod> <contenedor> -- <terminal>

El terminal puede ser /bin/bash o /bin/sh (si el contenedor no tiene instalado bash)
-it indica hacer el comando interactivo y abrir un terminal

Comandos para pods

Con kubectl cp copiamos archivos desde y a un contenedor:

kubectl cp <source> <destination> -c <container>

Para copiar DESDE un contenedor:

kubectl cp <nombre pod>:<ruta en el contenedor> <ruta en local> -c <container>

Para copiar A un contenedor:

kubectl cp <ruta en local> <nombre pod>:<ruta en el contenedor> -c <container>

Comandos para pods

kubectl cp puede copiar también directorios. La sintaxis es igual que rsync.

Un aspecto importante es que para que funcione el comando kubectl cp, en el contenedor tiene que estar instalado el paquete **tar**, ya que comprime los archivos antes de enviarlos y los descomprime en el destino, automáticamente.

Plugins

Desde hace varias versiones kubectl tiene un sistema de plugins. Cualquiera puede crear sus propios comandos de kubectl para automatizar las tareas más cotidianas. Seguramente haya uno para la tarea que buscas.

Krew es un gestor de plugins para Kubernetes, facilita su instalación y además tiene un directorio de plugins:

Krew: https://krew.sigs.k8s.io/

Directorio de plugins: https://github.com/kubernetes-sigs/krew-index

Plugins

También pueden instalarse a mano, en la documentación de cada plugin vienen las instrucciones.

Con kubectl plugin list podemos ver los plugins que tenemos instalados y las versiones.

Para utilizar un plugin sólo tenemos que ejecutar kubectl <comando del plugin> <parámetros>. De nuevo, cada plugin es distinto y lo mejor es ver su documentación.

¡Fin del módulo!

Hemos visto prácticamente todos los comandos que ofrece kubectl. Al menos hemos visto los que vas a utilizar seguro en tu día a día.

- Lo primero, hemos aprendido a instalarlo y configurarlo con el archivo kubeconfig y con comandos.
- Después hemos visto como podemos listar cualquier tipo de recurso, y a filtrar y personalizar el formato de salida.
- También algunos atajos para crear recursos fácilmente cuando lo necesitemos. Y para crear YAMLs rápidamente.
- Comandos para escalar, crear services y hacer rollback para deployments, statefulsets y daemonsets.
- Y comandos del día a día para interactuar con los pods.

Extras

Páginas de la documentación oficial:

Instalar y configurar kubeclt
kubectl CheatSheet
Referencia kubectl

¡Gracias!